KUBERNETES COMPREHENSIVE · AXP-K8S-5D

Day 1 Foundations & First Deployment

Participant Manual

Platform	AxisPay (fictional)
Kubernetes	v1.36
Environment	Ubuntu 26.04 LTS · Minikube
Built	14 August 2026

Every merchant, customer, card token, acquirer and transaction in this manual is fictional. No real institution is represented and no card number exists anywhere in this platform — only tokens. The platform is **not** PCI-DSS compliant and is not presented as such; it uses PCI-shaped constraints to make security controls consequential in a training context.

Day 4 — Networking, Exposure and Placement

AxisPay · Kubernetes Comprehensive · Participant Manual, Chapter 4

What changed today

Yesterday	Today
Reachable only via `port-forward`	Two hostnames over TLS
Flat pod network — the DMZ could read the database	22 NetworkPolicies, zero trust
Replicas placed by luck	One per node, by instruction
A drain evicted everything at once	Six PodDisruptionBudgets
DNS a black box	`ndots`, search domains and CoreDNS opened up

4.1 The cluster network model

1. What it is

A set of rules Kubernetes requires any network plugin to satisfy, plus the plugin (Calico, Cilium, Flannel) that actually satisfies them.

2. Why it exists

Kubernetes deliberately does not implement networking. It defines a contract — the **CNI** — so the same manifests run on a laptop, a data centre and three different clouds.

3. The business problem

AxisPay's services must reach each other predictably across nodes, and merchants must reach the platform from outside. Neither is possible without knowing what the network guarantees.

4. How it works

The four rules:

1. Every pod gets its own IP address — not a port on a shared host IP.
2. Pods reach all pods **without NAT**, across nodes.
3. Nodes reach all pods without NAT.
4. The IP a pod sees itself as is the IP others see it as.

> Rule 4 sounds obvious and is not true of Docker's default bridge, where a container sees `172.17.0.x` while the world reaches it on a mapped host port. A container there cannot truthfully tell anyone its own address. Kubernetes forbids that, which is why service discovery is straightforward here.

5. Internal architecture

```
pod A  --> node kernel --> CNI (Calico) --> node kernel --> pod B
                           routes/overlay
```

Calico can operate in **BGP mode** (real routes, no encapsulation, fastest) or **VXLAN/IPIP mode** (encapsulation, works anywhere). Minikube uses the latter.

Crucially, connectivity and policy enforcement are separate features. A plugin can satisfy all four rules and implement no NetworkPolicy at all — in which case every policy you write applies cleanly and enforces nothing.

6. Component interactions

```
kubelet -> CNI ADD -> plugin assigns an IP, wires the veth pair
kube-proxy -> watches EndpointSlices -> programs iptables/IPVS
Calico Felix -> watches NetworkPolicy -> programs additional rules
```

7. Enterprise example

A bank runs Calico in BGP mode peering with the physical network, so pod IPs are routable from outside the cluster. That makes firewall rules and traffic capture behave normally — worth a great deal to a security team, and impossible with an overlay.

8. Real-world analogy

A postal system. Every address is unique and reachable directly, and the letter arrives with the real sender's address on it. Contrast with a company switchboard that rewrites every outbound number — the recipient cannot call you back.

9. Best practices

Practice	Reason
Choose a CNI that enforces NetworkPolicy	Otherwise every policy you write is decoration
Choose it at cluster creation	CNI cannot be changed on a running cluster
Understand overlay vs routed mode	Overlay costs MTU and a little CPU; routed needs network cooperation
Watch MTU	An overlay reduces the usable MTU; mismatches cause slow, intermittent failures

10. Common mistakes

Mistake	Symptom
Assuming the default CNI enforces policy	Policies apply, nothing is enforced, no error
Ignoring MTU with an overlay	Large payloads hang; small ones work
Expecting pod IPs to be stable	They are not — that is what Services are for
Assuming pod IPs are routable outside	Only in routed mode

11. Security considerations

Threat	Control	Residual risk
Lateral movement across a flat network	NetworkPolicy + a policy-capable CNI	Requires the right plugin
Traffic sniffing between pods	Encryption (WireGuard in Calico, mTLS in a mesh)	Adds operational cost
Pod IP spoofing	Most CNIs enforce source addresses	Depends on the plugin

12. Performance considerations

- **Overlay encapsulation** costs a few percent CPU and reduces MTU (typically 1450 vs 1500).
- **iptables mode** in kube-proxy is O(n) in rule count and degrades past a few thousand Services; **IPVS** uses hash tables.
- **DNS is frequently the real latency cost** — see §4.2.

13. High availability

The network is per-node and inherently distributed. Failure modes: a CNI DaemonSet pod unhealthy on one node (pods there cannot get IPs — the node goes `NotReady`), or a control-plane outage stopping *changes* while existing traffic continues.

14. Disaster recovery

CNI configuration lives on the node and in the plugin's own CRDs. Losing it means pods cannot get addresses. Back up Calico's IP pool configuration alongside etcd.

15. Monitoring

Metric	Why
`kube_node_status_condition{condition="NetworkUnavailable"}`	CNI failure on a node
CNI DaemonSet ready count	Should equal the node count
`kubeproxy_sync_proxy_rules_duration_seconds`	Rising = too many Services
Pod IP allocation failures	IP pool exhaustion

16. Troubleshooting

Symptom	Cause	Command	Fix
Node `NotReady`, `NetworkUnavailable`	CNI pod unhealthy	`kubectl get pods -n kube-system -l k8s-app=calico-node -o wide`	Restart the CNI pod
Pod stuck `ContainerCreating`	CNI cannot assign an IP	`kubectl describe pod` → Events	Check the IP pool
Cross-node traffic fails, same-node works	Overlay/routing broken	Test with pod IPs directly	Check the CNI and MTU
Large requests hang, small ones work	**MTU mismatch**	`ping -M do -s 1400` between pods	Align MTU
NetworkPolicies do nothing	CNI does not enforce them	`kubectl get ds -n kube-system`	Rebuild with a policy-capable CNI

Interview questions

1. **What are the four networking rules Kubernetes requires?** *Every pod has its own IP; pods reach all pods without NAT; nodes reach all pods without NAT; the IP a pod sees itself as is the one others see.*
2. **Does Kubernetes implement networking?** *No. It defines the CNI contract and a plugin implements it. That is why the plugin choice matters and why it cannot be changed on a running cluster.*
3. **Your NetworkPolicies apply cleanly and traffic still flows. Why?** *(senior) The CNI does not implement policy enforcement. Connectivity and policy are separate features; Kubernetes stores the object regardless. Verify with a policy-capable CNI such as Calico or Cilium.*

4.2 DNS and service discovery

1. What it is

CoreDNS: a cluster DNS server that answers queries for Services and pods.

2. Why it exists

Pod IPs change constantly. Services provide a stable IP; DNS provides a stable *name* so configuration does not carry addresses.

3. The business problem

Every AxisPay manifest configures downstream services by name. Those names must resolve in every namespace, and the resolution must be fast enough to sit inside a 300 ms budget.

4. How it works

Every pod gets `/etc/resolv.conf`:

```
nameserver 10.96.0.10
search axispay-core.svc.cluster.local svc.cluster.local cluster.local
options ndots:5
```

Form	Resolves from
`payment-service`	Same namespace only
`payment-service.axispay-core`	Anywhere
`payment-service.axispay-core.svc.cluster.local`	Anywhere — fully qualified
`postgres-0.postgres.axispay-data.svc.cluster.local`	A **specific pod** of a StatefulSet

`ndots:5` is the performance trap. A name with fewer than five dots gets every search suffix appended *before* being tried as an absolute name. `payment-service.axispay-core.svc` has three dots, so the resolver tries up to four wrong names first. A **trailing dot** makes it absolute and skips the search list entirely.

5. Internal architecture

CoreDNS is a Deployment (usually 2 replicas) behind a ClusterIP Service. It watches the API server for Services and EndpointSlices and answers from memory. Its behaviour is configured by the **Corefile** in a ConfigMap — and a typo there crash-loops both replicas, which is INC-4b.

6. Component interactions

```
pod -> resolv.conf -> kube-dns ClusterIP -> (kube-proxy rules) -> CoreDNS pod
CoreDNS -> watches API server -> answers from cache
```

Note the recursion: reaching CoreDNS depends on kube-proxy having programmed the rules for the `kube-dns` Service.

7. Enterprise example

A high-throughput platform deploys **NodeLocal DNSCache** — a DaemonSet caching DNS on every node — cutting p99 lookup latency and removing CoreDNS as a hot path. They also set `ndots: 2` cluster-wide and use FQDNs everywhere.

8. Real-world analogy

A phone book plus a habit of trying extensions before full numbers. `ndots:5` is that habit: for short names it tries every local prefix before dialling the number as given.

9. Best practices

Practice	Reason
Use FQDNs in configuration	Removes cross-namespace ambiguity and avoids search-list overhead
Allow both UDP and TCP on 53 in policies	TCP is the fallback for large responses; UDP-only fails intermittently
Run at least 2 CoreDNS replicas	It is on every request path
Consider NodeLocal DNSCache at scale	Removes a hot dependency
Watch CoreDNS resource usage	It is CPU-bound under heavy query load

10. Common mistakes

Mistake	Symptom
Short names across namespaces	Resolution failure
Allowing only UDP/53 in a policy	**Intermittent** failures — much harder to diagnose
Forgetting DNS egress after default-deny	Everything fails with a resolution error
Ignoring `ndots`	Three wasted round-trips per call at scale
Single CoreDNS replica	One restart takes out cluster-wide resolution

11. Security considerations

Threat	Control	Residual risk
DNS enumeration reveals architecture	NetworkPolicy; DNS is broadly readable in-cluster	Hard to prevent entirely
DNS spoofing	Cluster DNS is internal; use mTLS for real authentication	—
DNS as an exfiltration channel	Egress policy restricting external DNS	Internal DNS must stay open

12. Performance considerations

- `ndots:5` costs up to **four extra lookups** per short name.
- CoreDNS caches; a cold cache after a restart shows as a latency spike.
- DNS failures often appear *gradually* because cached entries expire at different times — which is exactly why INC-4b presents as "intermittent".

13. High availability

Two replicas, anti-affinity across nodes, and a PDB. NodeLocal DNSCache adds a per-node layer so a CoreDNS outage degrades rather than breaks.

14. Disaster recovery

CoreDNS is stateless; recovery is redeploying it. The Corefile is the thing to back up — and to review carefully, since a typo takes out cluster-wide resolution.

15. Monitoring

Metric	Threshold
`coredns_dns_responses_total{rcode="SERVFAIL"}`	Any sustained rate
`coredns_dns_request_duration_seconds` **p99**	> 50 ms
CoreDNS ready replicas	< 2
`coredns_cache_hits_total` **/ misses**	Falling hit rate = cache pressure

16. Troubleshooting

Symptom	Cause	Command	Fix
Nothing resolves	CoreDNS down	`kubectl get pods -n kube-system -l k8s-app=kube-dns`	Restart; check the Corefile
Short name fails cross-namespace	Expected	—	Use the FQDN
Intermittent failures	Only UDP/53 allowed, or CoreDNS flapping	`kubectl get netpol -o yaml`; CoreDNS logs	Allow TCP/53
Slow DNS under load	`ndots:5` with short names	`time nslookup`	FQDNs, trailing dots, or NodeLocal cache
Resolution fails only after a policy change	DNS egress blocked	`kubectl get netpol`	Add `allow-dns-egress`

Interview questions

1. **What does `ndots:5` do?** *Any name with fewer than five dots has the search suffixes appended before being tried as absolute — up to four wasted lookups. A trailing dot skips the search list.*
2. **Why allow both UDP and TCP on port 53?** *DNS uses UDP normally and falls back to TCP for responses over 512 bytes. Allowing only UDP produces intermittent failures that are far harder to diagnose than a total outage.*
3. **You apply a default-deny NetworkPolicy and everything breaks. Why?** *(senior) Every service call begins with a DNS lookup to CoreDNS in `kube-system`, and that lookup is egress traffic. The symptom is name resolution failure, not connection refusal, so it looks like a DNS outage rather than a policy problem.*

4.3 Ingress

1. What it is

Two things: an **Ingress resource** describing HTTP routing rules, and an **Ingress controller** that reads those rules and proxies traffic.

2. Why it exists

A LoadBalancer Service per application is expensive and gives you no HTTP-level routing. Ingress puts one entry point in front of many services, with host and path routing plus TLS.

3. The business problem

Kalahari Coffee Roasters go live on Monday and will call `https://api.axispay.local/api/v1/charges` from their own servers. `kubectl port-forward` is not a product, and card data cannot travel over plaintext HTTP.

4. How it works

```
merchant --TLS:443--> Ingress controller --http:8080--> Service --> pods
                            ^
                      reads Ingress resources
```

An Ingress with no controller is a document nobody reads. It appears in `kubectl get ingress` with an **empty ADDRESS**, and nothing works. That empty column is the diagnostic.

`pathType` decides matching:

Type	`/api/v1` matches
`Prefix`	`/api/v1`, `/api/v1/charges`, everything below
`Exact`	**only** the literal `/api/v1`
`ImplementationSpecific`	controller-dependent — avoid

5. Internal architecture

`ingress-nginx` runs as a Deployment, watches Ingress resources, generates an nginx configuration and reloads. `ingressClassName` decides which controller claims a resource — essential on a cluster with more than one.

Controller-specific behaviour arrives via **annotations** (`limit-rps`, `proxy-read-timeout`, `ssl-redirect`), which is the main criticism of Ingress: the portable part is small and the useful part is vendor-specific. **Gateway API** is the successor and addresses exactly this.

6. Component interactions

```
you        -> create Ingress
controller -> watch -> regenerate nginx.conf -> reload
merchant   -> TLS to controller -> terminate -> proxy to Service ClusterIP
Service    -> EndpointSlice -> pod
```

7. Enterprise example

A payments platform runs two ingress controllers: an internet-facing one for merchant APIs with strict rate limits and WAF integration, and an internal one for staff tools. `ingressClassName` separates them, and certificates are issued by cert-manager against an ACME issuer with automatic rotation.

8. Real-world analogy

A building receptionist. One front door, one set of rules about who goes where, and one place where identity is checked. Without a receptionist the door still exists — nobody is directing anyone.

9. Best practices

Practice	Reason
Always set `ingressClassName`	Otherwise nobody claims it, or two controllers both do
Use `pathType: Prefix` unless you mean exact	`Exact` matches one literal path
Terminate TLS at the Ingress	One place for certificates
Use cert-manager in production	Manual certificates expire — and INC-7 is exactly that
Rate limit at the edge	Cheaper than at the application
Never expose `/healthz` externally	Probe endpoints are internal
Read the controller's logs when debugging	They record the routing decision

10. Common mistakes

Mistake	Symptom
No controller installed	Empty ADDRESS, nothing works, no error
`pathType: Exact` **by accident**	404 on every endpoint but one — **INC-4a**
Wrong backend port name	502
Backend Service with no endpoints	503
Forgetting the NetworkPolicy for the controller	503 after applying default-deny
Certificate expiring unnoticed	Total outage — **INC-7**

11. Security considerations

Threat	Control	Residual risk
Plaintext card data	TLS termination + `ssl-redirect`	TLS ends at the Ingress; use mTLS inside for defence in depth
Volumetric abuse	Rate limiting annotations	A distributed flood needs upstream help
Certificate expiry	cert-manager with automatic renewal	Monitoring is still required
Exposing internal endpoints	Explicit path rules; never wildcard	Review changes carefully

12. Performance considerations

- TLS termination is CPU-bound; scale the controller with traffic.
- **nginx reloads on every Ingress change** — thousands of Ingresses make reloads slow.
- `proxy-read-timeout` must exceed your slowest legitimate request; the reporting Ingress uses 120 s against the API's 30 s.

13. High availability

Run several controller replicas with anti-affinity and a PDB. In cloud environments they sit behind a LoadBalancer Service. The controller is a single point of failure for **all** external traffic — treat it accordingly.

14. Disaster recovery

Ingress resources are configuration and recover from Git. **Certificates are the hard part**: manual ones must be backed up and restored, which is precisely why cert-manager is the production answer. Also note that recreating a cloud LoadBalancer usually allocates a **new IP**, breaking DNS and any merchant allow-lists.

15. Monitoring

Metric	Threshold
`nginx_ingress_controller_requests{status=~"5.."}` **rate**	Any increase
`nginx_ingress_controller_ssl_expire_time_seconds`	< 30 days — **page**
Controller ready replicas	< 2
`nginx_ingress_controller_config_last_reload_successful`	0 = bad configuration

16. Troubleshooting

Symptom	Cause	Command	Fix
ADDRESS empty	No controller, or wrong class	`kubectl get ingressclass`	Install; set `ingressClassName`
404	Path or `pathType` wrong	Controller logs	Use `Prefix`
502	Wrong backend port	`kubectl describe ingress`	Match the Service port name
503	Backend has no endpoints	`kubectl get endpointslice`	Fix the Service or readiness
503 only after policy changes	Controller blocked by NetworkPolicy	`kubectl get netpol -n <ns>`	Allow the `ingress-nginx` namespace
TLS warning	Self-signed	`openssl s_client`	cert-manager

> **404, 502 and 503 point at three different layers.** Knowing which saves ten minutes every time.

Interview questions

1. **What is the difference between an Ingress resource and an Ingress controller?** *The resource is a document describing rules; the controller is a program that reads it and proxies. Creating a resource with no controller changes nothing — `kubectl get ingress` shows an empty ADDRESS.*
2. **`Prefix` versus `Exact`?** *`Prefix` matches the path and everything below it. `Exact` matches only the literal path. Changing one word turns a working API into a 404 on every endpoint but one.*
3. **You get 404, then 502, then 503 from an Ingress. What does each tell you?** *(senior) 404: the routing rules did not match — an Ingress problem. 502: the controller reached a backend and got a bad response, usually the wrong port. 503: no ready endpoints behind the Service — a Service or readiness problem. Three codes, three layers.*

4. **What does Gateway API fix?** *(senior) It replaces controller-specific annotations with typed resources, separates infrastructure and application concerns into different objects with different RBAC, and supports protocols beyond HTTP. Ingress's portable surface is small and its useful surface is vendor-specific; Gateway API addresses both.*

4.4 NetworkPolicy

1. What it is

A namespaced object describing which traffic is permitted to and from a set of pods.

2. Why it exists

By default every pod can reach every other pod in the cluster. In a regulated environment that is a finding, not a feature.

3. The business problem

Yesterday you connected to PostgreSQL from `edge-gateway`. In a PCI assessment that puts the DMZ inside the **cardholder data environment** — so every CDE control applies to the gateway, to everyone who can deploy it, and to its logs. The audit gets larger, longer and more expensive.

4. How it works

Three properties decide everything:

1. **Default-allow until selected.** A pod with no policy selecting it is unrestricted. The moment *any* policy selects it, only what a policy explicitly permits is allowed.
2. **Additive — there is no deny rule.** Policies only add permission. You deny by selecting a pod and permitting nothing.
3. **Both directions, independently.** Egress at the source *and* ingress at the destination must both allow a flow — two objects, often in two namespaces.

Default-deny is therefore just:

```
spec:
  podSelector: {}                  # every pod
  policyTypes: [Ingress, Egress]   # both directions
  # ...and no rules at all
```

Enforcement is the CNI's job. Kubernetes stores the object whatever your plugin does.

5. Internal architecture

AxisPay's 22 policies, in the order they are applied:

#	Policy	Effect
1	`default-deny-all` × 4 namespaces	Everything stops
2	`allow-dns-egress` × 4	UDP **and** TCP on 53 to CoreDNS
3	`allow-edge-to-payment` / `-merchant`	The DMZ may enter the CDE, narrowly
4	`allow-payment-to-core-services`	Only `payment-service` may call fraud/routing/ledger/customer
5	`allow-core-to-data`	Core reaches PostgreSQL, Redis, RabbitMQ
6	`allow-core-and-async-to-data`	The vault accepts core and async — **not edge**
7	`allow-ingress-controller`	ingress-nginx may reach the gateway
8	`allow-prometheus-scrape` × 3	Observability may scrape; nothing may reach it

> **The most important policy is the one that does not exist.** Nothing grants `axispay-edge` access to `axispay-data`. That omission is the control.

6. Component interactions

```
you    -> create NetworkPolicy
API    -> store
Calico Felix -> watch -> program iptables/eBPF on every node
packet -> evaluated at BOTH ends: egress at source, ingress at destination
```

7. Enterprise example

A bank runs default-deny in every namespace, generated automatically at namespace creation. Policy changes require a review from the security team, and the CI pipeline runs a policy simulator against a fixed list of must-allow and must-block flows before merge. That simulator is `platform/admin/validate/simulate-netpol.py` in this repository.

8. Real-world analogy

Building access control where **every door is locked by default** and each badge lists exactly which doors it opens. There is no "deny" badge — access is the absence of permission.

Where it breaks: a building has one access system. Kubernetes evaluates the door you leave *and* the door you enter, independently, and both must permit you.

9. Best practices

Practice	Reason
Default-deny first, then allow-list	Building the allow-list first looks strict and enforces nothing
Always allow DNS egress explicitly	Otherwise everything fails with a resolution error
Allow both UDP and TCP on 53	UDP-only gives intermittent failures
Write egress rules, not just ingress	Ingress stops attackers getting in; egress stops data getting out
Test enforcement empirically	Reading YAML proves nothing
Simulate in CI	A silently non-enforcing policy set is invisible
Label namespaces deliberately	`namespaceSelector` matches labels, not names

10. Common mistakes

Mistake	Symptom
Forgetting DNS egress	Everything fails; looks like a DNS outage
Only allowing UDP/53	Intermittent failures
Writing only ingress rules	No exfiltration protection
Assuming a policy denies	Policies only add; absence denies
Adding a narrow policy to an open pod	Everything else to that pod is now blocked — **INC-4c**
CNI without enforcement	Applies cleanly, protects nothing
Deleting a policy to fix an outage	Service restored, control removed, audit failed

11. Security considerations

Threat	Control	Residual risk
Lateral movement after a compromise	Default-deny + narrow allow-list	A compromised pod can still use its own permitted paths
Data exfiltration	Egress rules	DNS egress must stay open and is a known covert channel
DMZ reaching cardholder data	No policy grants it — the omission	Someone can add one
Policy silently not enforced	Verify the CNI; simulate in CI	Requires discipline

12. Performance considerations

Policies become iptables or eBPF rules. Very large policy sets increase rule-evaluation cost; Calico's eBPF mode scales better than iptables. Policy changes are pushed to every node, so churn costs CPU across the cluster.

13. High availability

Policies are enforced per node by the CNI agent. If that agent is unhealthy on a node, enforcement there may be stale — which is a **security** failure rather than an availability one, and far less visible. Monitor CNI agent health as a security control.

14. Disaster recovery

Policies are configuration and recover from Git. The operational risk is the opposite of loss: someone **deleting** a policy during an incident to restore service, and it never being restored. Any policy change made under pressure must be reviewed afterwards.

15. Monitoring

Signal	Why
Calico denied-packet metrics	Shows what policy is dropping
CNI agent ready count	Stale enforcement is silent
Policy count per namespace	A sudden drop means someone deleted one
Simulator result in CI	Catches regressions before merge

16. Troubleshooting

Symptom	Cause	Command	Fix
Everything fails after default-deny	DNS egress missing	`nslookup` from a pod	Add `allow-dns-egress`
Intermittent failures	UDP-only DNS rule	`kubectl get netpol -o yaml`	Allow TCP/53
Applied, nothing enforced	CNI lacks policy support	`kubectl get ds -n kube-system`	Rebuild with Calico
Traffic blocked, no error anywhere	Packets dropped, not refused	`nc -z -w3` from a pod	Read the policies — **INC-4c**
Policy looks right, still blocked	Only one direction allowed	Check **both** namespaces	Both must permit
`namespaceSelector` matches nothing	Namespace missing the label	`kubectl get ns --show-labels`	Label it

Interview questions

1. **Is NetworkPolicy default-allow or default-deny?** *Default-allow, until at least one policy selects the pod. From then on, only explicitly permitted traffic is allowed.*
2. **How do you write a deny rule?** *You cannot. There is no deny. You ensure some policy selects the pod and that none of them permit the traffic. Absence of permission is the denial.*
3. **You apply default-deny and everything breaks, including calls you allowed. Why?** *DNS. Every service call starts with a lookup to CoreDNS, which is egress traffic. The symptom is name resolution failure, so it reads as a DNS outage rather than a policy problem.*
4. **A narrow policy is added to one service and a completely different service breaks. Explain.** *(senior) Policies are additive and default-allow-until-selected. Before the policy, the target pod accepted traffic from anywhere. Adding one narrow policy makes it accept traffic ONLY from what that policy permits — everything else is now denied, including callers never mentioned in it. That is INC-4c.*
5. **A QSA asks you to demonstrate that the DMZ cannot reach cardholder data. What do you show?** *(senior) A live connection attempt from a DMZ pod to the database that times out, the policy set showing no rule granting that path, and a CI-run simulator asserting it. Evidence is a failed connection and a repeatable test — not a YAML file.*

4.5 Placement and disruption

1. What it is

Controls over **where** pods run (nodeSelector, affinity, taints) and **how many** may be voluntarily removed at once (PodDisruptionBudget).

2. Why it exists

Three replicas on one node is not redundancy — it looks like it and is not. And a node drain with no budget evicts everything on it simultaneously.

3. The business problem

Friday's capstone upgrades AxisPay under live merchant traffic, and part of that is draining a node. Without budgets that is a visible outage.

4. How it works

Three mechanisms, three directions:

Mechanism	Direction
`nodeSelector` / `nodeAffinity`	The **pod** is attracted to certain **nodes**
`podAffinity` / `podAntiAffinity`	The **pod** is attracted to / repelled by **other pods**
taints / tolerations	The **node** repels **pods**

> A **toleration does not attract**. It only removes an objection. To repel everything else *and* attract a specific workload you need a taint on the node plus both a toleration and a `nodeSelector` on the pod.

`required` **vs** `preferred`**:**

	Behaviour	AxisPay uses it for
`requiredDuringScheduling...`	Hard filter. Unsatisfiable → **Pending forever**.	`payment-service` — one per node, absolutely
`preferredDuringScheduling...`	Scoring hint, best effort	`fraud-service` — its HPA scales past the node count

> **The trap:** `required` anti-affinity on hostname means you can never have more replicas than nodes. Combined with an HPA it silently caps autoscaling — and the symptom (pods Pending during a spike) looks nothing like the cause.

PodDisruptionBudget gates **voluntary** disruption only:

Event	PDB applies?
`kubectl drain`	**Yes**
Cluster/node-pool upgrade	**Yes**
Node crash	No
OOM kill	No
Liveness restart	No
`kubectl delete pod`	**No** — a direct delete bypasses the eviction API

5. Internal architecture

The scheduler runs **filter** then **score**. Filter removes nodes that cannot run the pod (resources, taints, `required` affinity, node selectors). Score ranks the survivors (spread, `preferred` affinity, image locality). If filtering leaves nothing, the pod stays `Pending`.

`topologySpreadConstraints` are more expressive than anti-affinity: `maxSkew` allows "roughly even within a tolerance" rather than binary same-node-or-not, and `whenUnsatisfiable` chooses between refusing and degrading.

6. Component interactions

```
drain -> cordon node -> evict pods one at a time
       -> eviction API consults the PDB
       -> if the budget would be breached: WAIT and retry
       -> replacement schedules elsewhere (affinity applies again)
```

7. Enterprise example

A bank spreads every production workload across three availability zones with `topologySpreadConstraints` on `topology.kubernetes.io/zone`, and gives every workload a PDB of `maxUnavailable: 1`. Node-pool rotations run continuously and are a non-event. Nodes hosting cardholder workloads are tainted so nothing else can land on them.

8. Real-world analogy

Staff rostering. Anti-affinity is "not everyone on the same shift". A taint is "this site requires a security clearance". A toleration is "I have the clearance" — which does not mean you are assigned there. A PDB is "at least four people must be on the floor at all times", which constrains planned leave and does nothing about someone calling in sick.

9. Best practices

Practice	Reason
`required` anti-affinity for critical paths	Real redundancy, not apparent
`preferred` for autoscaled workloads	`required` silently caps the HPA
Prefer `maxUnavailable` over `minAvailable`	An absolute `minAvailable` can block all drains at minReplicas
Give every production workload a PDB	Otherwise a drain takes everything
Document the *absence* of a budget	A future operator needs to know it is deliberate
Spread on zone, not just hostname	A hostname spread does not survive a zone failure
Never `maxUnavailable: 0`	The node becomes undrainable, so it cannot be patched

10. Common mistakes

Mistake	Symptom
`required` anti-affinity plus an HPA	Autoscaling silently caps at the node count
`minAvailable` equal to current replicas	Drain hangs forever
`maxUnavailable: 0`	Node can never be maintained
Toleration without `nodeSelector`	Pod tolerates the taint but is not attracted there
No PDB	Drain evicts everything at once
Expecting a PDB to prevent a crash	It gates voluntary disruption only

11. Security considerations

Threat	Control	Residual risk
Cardholder workloads on shared nodes	Taints + `nodeSelector` for a PCI node pool	Kernel is still shared
An attacker forcing eviction	RBAC on `pods/eviction`	Cluster-admin can always drain
PDB used to block maintenance	Alert on `disruptionsAllowed: 0`	Requires monitoring

12. Performance considerations

- `required` affinity increases scheduling time — every candidate node is checked.
- `podAffinity` is expensive at scale: it is O(pods) per scheduling decision.

- `percentageOfNodesToScore` trades placement quality for speed on very large clusters.

13. High availability

Placement is the mechanism by which replica count becomes availability. Three replicas on one node survive nothing. The full recipe: replicas ≥ 3 · anti-affinity or topology spread · a PDB · readiness probes · storage that survives node loss.

14. Disaster recovery

Placement rules are configuration. The DR-relevant question is whether your constraints are **satisfiable in a degraded cluster**: `required` anti-affinity across three nodes means losing one node leaves a pod permanently Pending. That is correct behaviour, and it must be understood before an incident rather than discovered during one.

15. Monitoring

Metric	Threshold
`kube_poddisruptionbudget_status_pod_disruptions_allowed`	0 — maintenance is blocked
`kube_pod_status_unschedulable`	Any sustained
Pods per node for a critical Deployment	> 1 means spread failed
Node `unschedulable` (cordoned)	Longer than a maintenance window

16. Troubleshooting

Symptom	Cause	Command	Fix
Pod Pending, "didn't match pod anti-affinity"	More replicas than nodes with `required`	`kubectl describe pod`	Use `preferred`, or add nodes
HPA stops at the node count	`required` anti-affinity	`describe` a Pending pod	Switch to `preferred`
Drain hangs forever	PDB allows zero disruptions	`kubectl get pdb`	Scale up, or relax the budget
Drain evicts everything at once	No PDB	`kubectl get pdb -A`	Apply one
Toleration added, pod does not move	A toleration does not attract	`kubectl get pod -o wide`	Add a `nodeSelector`
Replicas still stacked	Rule applied but not rolled out	`kubectl rollout status`	Trigger a rollout

Interview questions

1. **What is the difference between a taint and anti-affinity?** *Direction. A taint is the node repelling pods; anti-affinity is the pod repelling other pods. A toleration removes the node's objection but does not attract the pod — for that you also need a* `nodeSelector`*.*
2. **When would you use** `preferred` **rather than** `required` **anti-affinity?** *When the replica count can exceed the node count — anything behind an HPA.* `required` *means you can never have more replicas than nodes, so it silently caps autoscaling during exactly the spike it was meant to absorb.*
3. **Does a PodDisruptionBudget protect against a node crashing?** *No. It gates voluntary disruption only — drains and upgrades via the eviction API. A crash, an OOM kill, a liveness restart and even* `kubectl delete pod` *all bypass it.*
4. **Your PDB is** `minAvailable: 3` **and the HPA has scaled to 3. You drain. What happens?** *(senior) Nothing, forever. Zero disruptions are allowed, so the drain hangs and the node can never be maintained. A node that cannot be drained cannot be patched — which is why* `maxUnavailable` *is safer with autoscaling.*
5. **Design placement for a payment service that must survive an availability-zone failure.** *(senior)* `topologySpreadConstraints` *on* `topology.kubernetes.io/zone` *with* `maxSkew: 1` *and* `DoNotSchedule`*, at least one replica per zone,* `minReplicas` *high enough that losing a zone still leaves capacity, a PDB of* `maxUnavailable: 1`*, and storage that is either zone-redundant or replicated at the application level.*

4.6 Service Discovery from a Java Application's Perspective

1. What it is

The path a Java process takes from a hostname like `payment-service.axispay-core.svc.cluster.local` to a TCP connection: libc resolver -> pod `resolv.conf` -> CoreDNS -> Service VIP or pod IP -> JVM socket.

2. Why it exists

Kubernetes service discovery is usually explained from the cluster's perspective. Incidents, however, happen inside the runtime that consumes it. A Java application has its own DNS cache, connection pools and retry behaviour, and those choices can contradict Kubernetes' assumption that endpoints change constantly.

3. The business problem

AxisPay rolled `payment-service` from version `2026.08.14.1` to `2026.08.14.2` at 14:05. The Deployment progressed cleanly. Readiness went green. EndpointSlices updated. The old pods terminated. Yet `fraud-service` began logging `java.net.ConnectException: Connection refused` for a narrow slice of requests and kept doing so for several minutes after the rollout had finished.

Operations first suspected kube-proxy. Then they suspected a half-dead node. The real cause sat inside the JVM: `fraud-service` had resolved `payment-service.axispay-core.svc.cluster.local` once, cached the answer, and kept trying the old pod IP after that pod had already exited. Kubernetes had moved on. The Java process had not.

4. How it works

Inside a normal pod you will see:

```
cat /etc/resolv.conf
```

```
nameserver 10.96.0.10
search axispay-core.svc.cluster.local svc.cluster.local cluster.local
options ndots:5
```

The sequence for `payment-service.axispay-core.svc.cluster.local` is:

1. The JVM asks the operating system resolver for the name.
2. The resolver sends the query to the cluster DNS Service (`kube-dns` / CoreDNS).
3. CoreDNS answers from the Service and EndpointSlice state it watches from the API server.

4. The JVM stores the result in its own DNS cache.
5. Java opens a socket to the returned IP.

That fourth step is the trap. Kubernetes assumes names may point to different endpoints over time. Many JVM configurations historically assume DNS answers are fairly static and cache them for a very long time.

Setting	Meaning	Operational effect
`networkaddress.cache.ttl=-1`	Cache forever	Excellent for static infrastructure, terrible for dynamic endpoints
`networkaddress.cache.ttl=30`	Cache for 30 seconds	Better, but still enough to outlive a terminating pod
`networkaddress.cache.ttl=10`	Cache for 10 seconds	Common compromise for Kubernetes
`networkaddress.cache.negative.ttl=10`	Cache failed lookups briefly	Prevents hot-looping on missing names

Two ways to set it:

```
// java.security
networkaddress.cache.ttl=10
networkaddress.cache.negative.ttl=10
```

or:

```
JAVA_TOOL_OPTIONS="-Dsun.net.inetaddr.ttl=10 -Dsun.net.inetaddr.negative.ttl=10"
```

The exact default depends on JDK version and whether the old security-manager-based settings are in play. The safe operational assumption is this: **never trust the JVM default in Kubernetes. Set it explicitly.**

5. Internal architecture

```
Spring Boot thread
  -> InetAddress.getAllByName()
    -> JVM DNS cache
      -> OS resolver
        -> /etc/resolv.conf
          -> CoreDNS Service VIP
            -> CoreDNS pod
              -> Service / EndpointSlice data
```

There are therefore **three caches** to think about:

Layer	Cache?	Notes
JVM	Yes	Often the real incident source
CoreDNS	Yes	Usually short-lived and healthy
HTTP/gRPC client pool	Yes, indirectly	Existing sockets can outlive DNS answers

Even if DNS TTL is fixed, a connection pool may continue using already-open sockets to an endpoint until the socket fails. DNS freshness and connection freshness are related but not identical.

6. Component interactions

```
payment-service rollout
  -> old pod removed from EndpointSlice
  -> CoreDNS starts answering with only new endpoints
  -> new Java lookups are correct
  -> fraud-service JVM keeps stale cached answer
  -> TCP SYN to dead IP
  -> connection refused / timeout
```

Kubernetes did the right thing quickly. The client application kept a memory of the old world.

ClusterFirst DNS policy is what makes this whole experience feel natural in pods. It tells Kubernetes to inject cluster DNS as the first resolver for the pod, ahead of the node's external DNS settings. Without `ClusterFirst`, short service names would not resolve as cluster names at all unless you configured them manually. For almost every in-cluster Java workload, `dnsPolicy: ClusterFirst` is the correct answer.

7. Enterprise example

A regulated fintech running 700+ Spring Boot services standardises three JVM flags on every workload:

```
-Dsun.net.inetaddr.ttl=10 \
-Dsun.net.inetaddr.negative.ttl=5 \
-Djava.security.egd=file:/dev/urandom
```

The first two make service discovery compatible with rolling updates. The third shortens startup. They also require every application property containing a service hostname to use the full Kubernetes FQDN so behaviour is identical from every namespace and every test environment.

8. Real-world analogy

A receptionist who keeps a handwritten list of extensions. If the office moves desks every hour, the list must be refreshed frequently. Kubernetes updates the company directory immediately; the receptionist's private notebook does not.

Where it breaks: a real receptionist can be told verbally that someone moved. A JVM cannot. It will use the cached answer until the TTL expires or the process restarts.

9. Best practices

Practice	Reason
Set JVM DNS TTL explicitly	Defaults vary; incidents should not depend on JDK trivia
Use `networkaddress.cache.ttl=10` as a starting point	Short enough for rollouts, long enough to avoid DNS storms
Use fully qualified Service names in config	Avoids namespace ambiguity and `ndots` search overhead
Keep `dnsPolicy: ClusterFirst` for normal workloads	Ensures cluster Services resolve correctly
Tune connection pools alongside DNS	Fresh DNS does not close long-lived stale sockets
Expose DNS settings in runtime diagnostics	Support teams need to see the effective JVM values quickly

A practical Spring Boot pattern is to print the effective TTL at startup and include the downstream FQDNs in the configuration report, without printing secrets.

10. Common mistakes

Mistake	Symptom
Relying on JVM default DNS TTL	Works in test, flakes during rollouts in prod
Using bare `payment-service` across namespaces	Resolves only in the same namespace
Assuming a Service name points to one stable pod	Connection refused after pod churn
Fixing DNS TTL but ignoring keep-alive sockets	Traffic still sticks to dead or overloaded endpoints
Using short external names with `ndots:5`	Slow first connection, especially on cold start

`ndots:5` matters here too. A short name such as `payment-service` or even `payment-service.axispay-core` is tried with the search list before it is treated as absolute. That means several failed lookups can happen before the successful one. Using `payment-service.axispay-core.svc.cluster.local` skips the ambiguity and removes the latency tax.

11. Security considerations

Threat	Control	Residual risk
DNS poisoning inside the cluster	Trust CoreDNS only; restrict arbitrary DNS egress	DNS is not authentication
Service confusion across namespaces	FQDNs in config	Humans can still misconfigure names
Stale cached answers after a failover	Short TTL + retries + mTLS identity	Short TTL increases query volume
External DNS exfiltration	Egress policy allowing only approved DNS paths	CoreDNS still remains broadly reachable

A Java client should authenticate the server at TLS or mTLS layer. DNS tells it *where* to go, not *who* it found.

12. Performance considerations

- A TTL of 0 is technically fresh and operationally expensive. Every request path now depends on DNS latency.
- A TTL of 10 seconds is often the sweet spot for Java services behind Kubernetes Services.
- `ndots:5` can add several failed queries before the real one, especially for names with fewer than five dots.
- FQDNs reduce wasted lookups and make cold-start latency more predictable.
- Negative caching matters too: if a dependency is temporarily absent, caching that miss for a few seconds protects CoreDNS from retry storms.

13. High availability

Service discovery is available when **all** of these hold:

1. CoreDNS has enough replicas.
2. The `kube-dns` Service routes correctly.
3. Clients do not cache stale answers indefinitely.
4. Clients retry or reconnect when endpoints change.

This is why "CoreDNS has two replicas" is not a full HA story. The consumer runtime must participate.

14. Disaster recovery

The DR question is not only "can CoreDNS recover?" but also "what happens to long-lived Java processes after a failover?" If `payment-service` is restored onto new pods after a node loss, clients with a permanent DNS cache can continue calling dead IPs until they restart. A post-failover runbook should therefore include validation from representative Java clients, not just from a BusyBox shell.

15. Monitoring

Signal	Why
`UnknownHostException` rate	DNS resolution failing entirely
`ConnectException: Connection refused` immediately after rollouts	Strong stale-endpoint indicator
CoreDNS request latency and error codes	Resolver health
Application startup logs showing effective TTL	Configuration drift detection
EndpointSlice churn during deploys	Helps correlate stale-cache incidents

A useful alert is: **rolling deployment of service X + sudden rise in connection refused from caller Y**. That pattern often points at client-side caching rather than server failure.

16. Troubleshooting

Symptom	Cause	Command	Fix
Calls fail for minutes after a clean rollout	JVM cached stale pod IP	`kubectl logs` for `ConnectException`; inspect JVM flags	Set TTL to 10 and roll clients
Same config works in one namespace, fails in another	Short Service name	`kubectl exec <pod> -- getent hosts payment-service`	Use the full FQDN
First call is slow, later ones fast	`ndots` + search-list expansion	`kubectl exec <pod> -- cat /etc/resolv.conf`	Use FQDNs or trailing dots
Resolution works, traffic still hits old backend	Connection pool holds socket open	Client metrics / connection pool stats	Shorten pool lifetime or force reconnect on failure
Service names do not resolve at all	Wrong DNS policy	`kubectl get pod -o yaml`	Use `dnsPolicy: ClusterFirst`

Interview questions

1. **Why can a Java service keep calling a dead pod after Kubernetes has already updated the Service endpoints?** *Because the JVM may cache the DNS answer longer than the pod lives. Kubernetes updates EndpointSlices and CoreDNS quickly, but the Java process may continue using its stale cached IP or an already-open socket.*
2. **What setting controls Java DNS caching in Kubernetes?** *`networkaddress.cache.ttl`, with `networkaddress.cache.negative.ttl` for failed lookups. In practice many teams set them explicitly via `java.security` or `-Dsun.net.inetaddr.ttl=10` rather than trusting the default.*

3. **Why prefer `payment-service.axispay-core.svc.cluster.local` over `payment-service`?** *It is unambiguous across namespaces and avoids `ndots` search expansion, which can add several failed DNS lookups before success.*
4. **What does `dnsPolicy: ClusterFirst` do?** *(senior) It tells Kubernetes to inject the cluster DNS Service as the pod's primary resolver so Service names resolve naturally inside the cluster. Without it, in-cluster service discovery becomes inconsistent or fails outright.*
5. **You set JVM TTL to 10 seconds and still see stale traffic. Why?** *(senior) Because DNS caching is only one layer. HTTP keep-alive or gRPC channels can keep using an existing socket long after DNS would have returned a new answer. You must consider connection lifetime, reconnect behaviour and retry policy as well as DNS TTL.*

4.7 gRPC and HTTP/2 communication behind Kubernetes Services

1. What it is

The interaction between Kubernetes Service load balancing and gRPC's HTTP/2 connection model.

2. Why it exists

A normal HTTP/1.1 client often opens many short-lived TCP connections. A Kubernetes Service spreads those connections reasonably well. gRPC over HTTP/2 does the opposite: it prefers one long-lived TCP connection carrying many multiplexed RPC streams. Kubernetes balances **connections**, not individual RPCs.

3. The business problem

AxisPay converted `fraud-service` to gRPC to reduce latency on risk scoring. Load tests looked fine with one caller. Production did not. `payment-service` opened a small number of long-lived channels to `fraud-service`, kube-proxy sent each channel to one backend pod, and one fraud pod climbed to 100% CPU while the other two sat nearly idle. Merchants saw sporadic 429-like application errors even though `kubectl get endpointslice` showed three healthy endpoints.

The service existed. The replicas existed. The traffic distribution did not.

4. How it works

A ClusterIP Service typically behaves like this:

```
client TCP connect -> kube-proxy chooses one backend pod
all bytes on that TCP connection -> same backend pod
connection closes -> next connect may choose a different pod
```

For HTTP/1.1 that may be acceptable. For gRPC it can be disastrous because many RPCs share one connection.

Protocol pattern	Connection shape	Service behaviour
HTTP/1.1 without keep-alive	Many short TCP connections	Distribution usually looks fair enough
HTTP/1.1 with keep-alive	Fewer, longer connections	Some pinning
gRPC / HTTP/2	Very few, very long connections carrying many RPCs	Severe pinning risk

That is why "we have a Service and three ready pods" does **not** prove that gRPC load is balanced.

The three standard fixes are:

1. **Headless Service + client-side load balancing.** The client resolves multiple pod IPs and chooses among them itself.
2. **Sidecar or node proxy at L7.** Envoy or Linkerd understands streams and can distribute requests intelligently.
3. **L7-aware gateway / ingress.** A proxy terminates or observes HTTP/2 and balances RPCs above the TCP layer.

5. Internal architecture

The simplest worked AxisPay example uses a headless Service:

```
apiVersion: v1
kind: Service
metadata:
  name: fraud-grpc
  namespace: axispay-core
spec:
  clusterIP: None
  selector:
    app: fraud-service
  ports:
    - name: grpc
      port: 9090
      targetPort: 9090
```

Now DNS returns multiple A records instead of one virtual IP. In `payment-service`, gRPC Java can be pointed at the DNS name and told to round-robin:

```
ManagedChannel channel = ManagedChannelBuilder
    .forTarget("dns:///fraud-grpc.axispay-core.svc.cluster.local:9090")
    .defaultLoadBalancingPolicy("round_robin")
    .usePlaintext()
    .build();
```

With that arrangement, the client library owns balancing across the resolved pod IPs rather than delegating everything to kube-proxy.

6. Component interactions

```
payment-service gRPC client
  -> resolve fraud-grpc headless Service
  -> CoreDNS returns pod IPs for all ready fraud-service pods
  -> grpc-java creates subchannels
  -> round_robin policy rotates RPCs across subchannels
  -> each pod receives a fairer share
```

Contrast that with the broken path:

```
payment-service
  -> connect once to fraud-service ClusterIP
  -> kube-proxy picks one fraud pod
  -> all RPC streams stay pinned there
```

7. Enterprise example

A large payments processor uses **Linkerd** for all east-west gRPC traffic. Every pod talks to a local sidecar over loopback. The sidecar maintains many upstream HTTP/2 connections and balances requests at L7 with latency-aware routing, retries and outlier detection. Application teams no longer need to understand gRPC resolver plugins or load-balancing policies; the platform owns it once.

That is more operationally complex than a headless Service, but it centralises behaviour and observability.

8. Real-world analogy

A hotel concierge assigning guests to lifts. Kubernetes chooses the lift **when you enter the lobby**. gRPC then keeps you inside that lift for the next hundred trips because it is reusing the same cabin. If one lift is slow, you suffer repeatedly while the others stay empty.

Where it breaks: real people can step out and pick another lift. An HTTP/2 client usually stays on the same connection until failure, idle timeout or an explicit rebalance.

9. Best practices

Practice	Reason
Assume ClusterIP balances connections, not gRPC calls	Prevents false confidence
Use headless Services for pure client-side gRPC balancing	DNS returns all pod IPs
Configure grpc-java with an explicit policy such as `round_robin`	Default behaviour may be `pick_first`
Watch per-pod CPU and request counts, not just Service-level totals	Pinning is invisible at Service level
Consider a sidecar or mesh for standardised L7 balancing	Centralises retries, TLS and metrics
Close and recreate channels deliberately during deploys if needed	Long-lived channels preserve imbalance

One subtle point: some clients use `pick_first` even when DNS returns several endpoints. That simply picks the first working address and stays there, which recreates the original problem under a different name.

10. Common mistakes

Mistake	Symptom
Putting gRPC behind a normal ClusterIP and assuming fairness	One pod hot, others cold
Using a headless Service without enabling client-side balancing	All traffic still sticks to one IP
Reading only Service-level metrics	Looks healthy while one pod melts
Reusing one channel forever	Imbalance persists across hours or days
Treating readiness as load health	Every pod can be ready and still underused

Connection pinning symptoms to watch for:

- one `fraud-service` pod at 100% CPU;
- one pod with almost all active RPC streams;
- other pods near idle despite being ready;
- errors disappear temporarily when the overloaded pod restarts, then return as the new channel pins again.

11. Security considerations

Threat	Control	Residual risk
East-west traffic interception	mTLS in mesh or application TLS	Client-side LB alone does not provide identity
Traffic concentration enabling easy DoS of one pod	L7 balancing + rate limits	A single client can still open many channels
Debugging with plaintext in prod	Use TLS-enabled gRPC channels	Certificate management overhead
Bypassing policy with direct pod IPs	NetworkPolicy still restricts allowed flows	Headless Services expose endpoint IPs to clients

A headless Service changes **how** endpoints are found, not **who** may reach them. NetworkPolicy still matters.

12. Performance considerations

- gRPC is efficient precisely because it reuses connections; that same efficiency causes pinning.
- Client-side round-robin usually reduces tail latency by preventing one hot pod from queueing all work.

- Sidecar proxies add a hop and some CPU, but often improve overall p99 by making traffic distribution sane.
- DNS refresh frequency matters: if the headless Service endpoint set changes, clients must re-resolve often enough to notice.

13. High availability

For gRPC, HA means more than replica count. A client pinned to one pod is not truly using the others for resilience. With client-side balancing or an L7 proxy, losing one fraud pod removes one subchannel; traffic drains to the others immediately. With `pick_first`, losing the chosen pod can cause a visible reconnect storm instead.

14. Disaster recovery

During a node loss, gRPC clients with headless DNS plus round-robin usually reconnect cleanly to the remaining pod IPs. During a full cluster failover, ensure the clients do not cache old endpoint sets forever and that DNS is re-resolved promptly. If a service mesh provides mTLS identities, its certificate and trust-chain restoration becomes part of the DR plan too.

15. Monitoring

Signal	Why
Per-pod QPS / RPC count	Reveals imbalance hidden by the Service
Per-pod CPU for `fraud-service`	One hot pod is the classic symptom
gRPC active stream count per channel	Shows connection concentration
Channel reconnect rate	Spikes during failures or aggressive rebalancing
DNS answer size / endpoint count for headless Service	Confirms the client sees all pods

A simple SRE dashboard panel that places per-pod RPC count next to total Service traffic catches this class of issue faster than any generic cluster dashboard.

16. Troubleshooting

Symptom	Cause	Command	Fix
One gRPC backend pod saturated, others idle	Connection pinning through ClusterIP	Compare per-pod CPU / request metrics	Use headless + client LB or L7 proxy
Headless Service created, no improvement	Client still using `pick_first`	Inspect client config / startup logs	Set `round_robin` explicitly
Clients fail after pod churn	DNS or channel set stale	`kubectl get endpointslice` ; client logs	Refresh resolver / recreate channels
Traffic random at startup then converges to one pod	One long-lived channel survives	Inspect channel count per instance	Use more channels or rebalance logic
NetworkPolicy blocks headless access	Pod IPs allowed path missing	`kubectl get netpol -n axispay-core`	Permit the caller to the backend port

Interview questions

1. **Why does kube-proxy load balancing often fail to distribute gRPC traffic evenly?** *Because kube-proxy chooses a backend when the TCP connection is created. gRPC multiplexes many RPCs over one long-lived HTTP/2 connection, so all those RPCs remain pinned to the same backend until the connection is recreated.*
2. **What is the simplest Kubernetes-native fix for gRPC connection pinning?** *A headless Service plus client-side load balancing. DNS returns multiple backend pod IPs and the gRPC client distributes across them.*
3. **What are the three standard fixes for gRPC behind Kubernetes?** *Headless Service with client-side balancing, a sidecar proxy such as Envoy or Linkerd doing L7 balancing, or an L7-aware ingress or gateway.*
4. **Why might a headless Service still not fix the problem?** *(senior) Because the client library may still use a `pick_first` policy or cache the first working endpoint. DNS returning many IPs is only step one; the client must actually balance across them.*
5. **How do you recognise connection pinning in production?** *(senior) One backend pod runs hot while others stay mostly idle, even though readiness is green and the Service has multiple endpoints. Per-pod CPU, per-pod RPC counts and active stream metrics show the skew immediately.*

4.8 NetworkPolicy Design for AxisPay's Service Mesh

1. What it is

A deliberate allow-list design for east-west traffic inside `axispay-core`, built from the real call graph rather than from guesswork.

2. Why it exists

Default-deny is the starting line, not the finish. A secure namespace still needs payments to flow, auth to validate tokens, fraud to score transactions and ledger to commit them. The difficult part is permitting exactly those flows and nothing adjacent.

3. The business problem

AxisPay split services into namespaces for operations and compliance, then turned on default-deny in `axispay-core`. The first smoke test failed everywhere. `auth-service` could not resolve names. `payment-service` could not reach `fraud-service`. The ledger path broke. Operators were tempted to delete the default-deny just to restore processing.

The better move was to map the real graph precisely: who initiates traffic, on which port, from which namespace, and what must never be reachable. Once that graph exists, policy design becomes engineering rather than superstition.

4. How it works

The pattern is always the same:

1. Deny everything for pods in the namespace.
2. Re-open DNS egress.
3. Add one policy per legitimate communication path.
4. Test from a live pod, not by reading YAML.

A minimal namespace baseline:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: axispay-core
spec:
  podSelector: {}
  policyTypes:
    - Ingress
    - Egress
```

Then the rule everyone forgets:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-dns-egress
  namespace: axispay-core
spec:
  podSelector: {}
  policyTypes: [Egress]
  egress:
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: kube-system
      ports:
        - protocol: UDP
          port: 53
        - protocol: TCP
          port: 53
```

From there, encode the real call graph.

5. Internal architecture

AxisPay's important application paths in `axispay-core` look like this:

Caller	Destination	Port	Why
ingress controller	`payment-service`	8080	Merchant traffic enters here
any service needing token checks	`auth-service`	8080	Token validation
`payment-service`	`fraud-service`	9090	gRPC risk scoring
`payment-service`	`core-service`	8080	Ledger orchestration
`core-service`	PostgreSQL in `axispay-data`	5432	Persistent ledger writes

That becomes several narrow policies rather than one broad "allow everything inside core" shortcut.

Example: allow ingress controller traffic to `payment-service`:

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-ingress-to-payment
  namespace: axispay-core
spec:
  podSelector:
    matchLabels:
      app: payment-service
  policyTypes: [Ingress]
  ingress:
    - from:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: ingress-nginx
      ports:
        - protocol: TCP
          port: 8080
```

Example: `payment-service` to `fraud-service` on gRPC:

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-payment-to-fraud
  namespace: axispay-core
spec:
  podSelector:
    matchLabels:
      app: payment-service
  policyTypes: [Egress]
  egress:
    - to:
        - podSelector:
            matchLabels:
              app: fraud-service
      ports:
        - protocol: TCP
          port: 9090
```

Example: allow the destination side as well:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-payment-into-fraud
  namespace: axispay-core
spec:
  podSelector:
    matchLabels:
      app: fraud-service
  policyTypes: [Ingress]
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: payment-service
      ports:
        - protocol: TCP
          port: 9090
```

The rule set is intentionally repetitive. Repetition is cheaper than ambiguity in security policy.

6. Component interactions

```
merchant -> ingress-nginx -> payment-service -> fraud-service
                                             \-> auth-service
                                             \-> core-service -> PostgreSQL
```

For each arrow, ask two questions:

1. Does the source pod's **egress** allow it?
2. Does the destination pod's **ingress** allow it?

If either answer is no, the packet disappears into a timeout.

7. Enterprise example

A card processor with hundreds of namespaces generates policies from a service catalog. Each service entry declares inbound ports, approved callers and approved outbound dependencies. CI renders NetworkPolicies from that catalog and then runs synthetic checks from ephemeral pods. The platform team treats the policy set like firewall code: reviewed, versioned, tested and diffed on every merge.

8. Real-world analogy

An airport with separate security zones. Default-deny locks every internal door. Then access cards are issued per route: check-in staff may enter passport control, baggage systems may enter the sorting area, finance may enter the vault. The absence of a card for a route is the control.

Where it breaks: airports usually care mostly about entry. Kubernetes cares equally about exit and entry; the source corridor and the destination door must both open.

9. Best practices

Practice	Reason
Start with `default-deny-all`	Otherwise narrow allow rules do not enforce anything
Add DNS egress immediately after	Prevents cluster-wide self-inflicted outage
Write policies from the call graph	Security follows architecture
Use labels consistently for `podSelector`	Policy is only as reliable as the labels
Permit by port and protocol, not just destination	Makes intent auditable
Test absent paths as well as present ones	"Must block" is as important as "must allow"
Separate ingress and egress rules cleanly	Easier incident reasoning

10. Common mistakes

Mistake	Symptom
Forgetting DNS egress	Every service appears down
Allowing only destination ingress, not source egress	Timeout despite "correct" policy
Writing one huge broad allow policy	Security goal collapses back to flat network
Matching namespace by name without labels	`namespaceSelector` never matches
Forgetting cross-namespace database rule	App reaches nothing in `axispay-data`
Removing default-deny during an incident	Outage ends, compliance control disappears

11. Security considerations

Threat	Control	Residual risk
Lateral movement from a compromised service	Narrow allow-list per call path	Compromised service can still use its approved paths
DNS-based break-glass by operators	Mandatory DNS rule codified once	CoreDNS remains broadly reachable
Payment service reaching arbitrary backends	Egress rules restricted to named dependencies	Human error in labels or ports
Database exposure to non-ledger services	Only `core-service` permitted to PostgreSQL	DB credentials still need separate protection

A useful governance rule is: **if a new dependency appears in code, it must appear in policy review too.** Architecture drift is often visible in network policy before it is visible in diagrams.

12. Performance considerations

- More policies mean more rule evaluation, but the cost is normally trivial compared with the value of segmentation.
- Coarse policies are cheaper and weaker; narrow policies are slightly costlier and far clearer.
- DNS denial creates the most confusing performance symptom: latency from repeated resolution failures rather than a clean hard reject.
- eBPF-based enforcement usually scales better than long iptables chains on very busy clusters.

13. High availability

A resilient policy design allows redundancy paths without over-opening the namespace. If `payment-service` has three replicas and `fraud-service` has three replicas, the policies should allow any payment pod to any fraud pod on the approved port. HA is not helped by pinning traffic to one named pod IP inside policy.

14. Disaster recovery

In DR, recovery means restoring both applications and segmentation. Rebuilding `axispay-core` without its policies may bring service back faster and fail the audit later. Store the policies in Git next to the manifests and validate that the restored cluster still blocks forbidden flows such as direct access from ingress-facing workloads to PostgreSQL.

15. Monitoring

Signal	Why
Denied packet counters by namespace	Shows which rule is missing or blocking
Count of policies in `axispay-core`	Sudden drop suggests accidental deletion
DNS success rate from app pods	Fastest indicator that baseline egress exists
Synthetic allow/block probes	Confirms behaviour, not intent
Cross-namespace connection failures during deploy	Catches missing `axispay-data` access early

16. Troubleshooting

Symptom	Cause	Command	Fix
Traffic works with NetworkPolicy absent but breaks when applied	Missing DNS egress	`kubectl exec <pod> -- nslookup kubernetes.default`	Add UDP and TCP 53 to `kube-system`
`payment-service` cannot call `fraud-service` after default-deny	Only ingress or only egress allowed	Compare both policies	Permit both source egress and destination ingress
`core-service` cannot reach PostgreSQL in `axispay-data`	Cross-namespace rule absent or selector wrong	`kubectl get netpol -n axispay-data -o yaml`	Add matching ingress on DB side and egress on core side
Ingress returns 503 after policies added	`ingress-nginx` namespace not allowed to `payment-service`	`kubectl get netpol -n axispay-core`	Allow ingress from the controller namespace
Policy appears correct, still no traffic	Labels do not match live pods	`kubectl get pods --show-labels`	Align selectors with actual labels
Some calls work, large ones fail strangely	DNS TCP fallback blocked	Inspect DNS policy	Allow TCP/53 as well as UDP/53

Interview questions

1. **What is the first policy you should create in a secure namespace, and what must follow it immediately?** *`default-deny-all`, followed immediately by DNS egress. Without the first, nothing is enforced; without the second, everything appears broken.*
2. **Why is DNS egress the policy people forget most often?** *Because DNS is infrastructure rather than business traffic. Every application call depends on it, but architects naturally think about app-to-app paths first and resolver traffic second.*
3. **Why must `payment-service -> fraud-service` usually be represented on both sides?** *Because NetworkPolicy evaluates source egress and destination ingress independently. If either side lacks permission, the packet is dropped.*
4. **What evidence proves a policy set is working?** *(senior) Live traffic tests showing approved paths succeed and forbidden paths fail, ideally automated in CI or synthetic monitoring. YAML alone proves intent, not enforcement.*
5. **How would you explain `traffic works with policy absent but breaks when applied` to a security reviewer?** *(senior) Absent policy means the pods were default-allow. Applying any selecting policy flips them into allow-list mode. The break indicates the allow-list is incomplete, most often missing DNS, one traffic direction, or a cross-namespace selector.*

4.9 DNS Debugging Toolkit

1. What it is

A repeatable runbook for proving whether a DNS problem lives in the pod, the namespace policy, CoreDNS, the Corefile, or the application runtime.

2. Why it exists

"DNS is broken" is usually too vague to fix. Kubernetes DNS failures can be caused by NetworkPolicy, CoreDNS crash loops, bad search-domain behaviour, stale client caches, or custom Corefile rewrite rules. A toolkit turns a panic into a sequence.

3. The business problem

`merchant-service` began timing out on its first webhook delivery to an external partner hostname, `webhooks.partner-payments.com`. Retries succeeded. CPU was normal. CoreDNS was healthy. Because only the **first** request was slow, the problem was written off as "internet weather" until it started burning into latency SLOs.

The real issue was local: the pod inherited `ndots:5`, treated the external hostname as non-absolute, tried the cluster search domains first, waited for several negative answers, and only then asked external DNS. The first connection paid that penalty every time the answer aged out of cache.

4. How it works

The runbook starts from the pod boundary.

A. Launch a throwaway debug pod

```
kubectl run dns-debug -n axispay-core --rm -it --restart=Never \
  --image=ghcr.io/nicolaka/netshoot:latest -- bash
```

Inside it, check the resolver:

```
cat /etc/resolv.conf
```

Then test several tools, because they exercise different layers:

```
nslookup payment-service.axispay-core.svc.cluster.local

dig payment-service.axispay-core.svc.cluster.local

getent hosts payment-service.axispay-core.svc.cluster.local
```

Tool	Best for
`nslookup`	Quick human-readable check
`dig`	Full answer, timings, search behaviour
`getent hosts`	What normal libc-based apps will see

B. Compare internal and external names

```
time getent hosts payment-service.axispay-core.svc.cluster.local

time getent hosts webhooks.partner-payments.com

time getent hosts webhooks.partner-payments.com.
```

That trailing dot matters. It marks the external hostname as fully qualified and bypasses the search list.

C. Inspect CoreDNS

```
kubectl get pods -n kube-system -l k8s-app=kube-dns
kubectl logs -n kube-system -l k8s-app=kube-dns --tail=100
kubectl get configmap coredns -n kube-system -o yaml
```

If someone added a `rewrite` rule or forwarder in the Corefile, read it carefully. A bad rule can make one name resolve somewhere surprising while everything else looks healthy.

5. Internal architecture

```
application -> libc / JVM resolver -> resolv.conf search list -> kube-dns Service
  -> CoreDNS Corefile plugins -> kubernetes plugin / rewrite / forward
  -> upstream DNS if external name
```

The key operational lesson is that DNS resolution is **plugin-based** in CoreDNS. The Corefile determines the order. A `rewrite` plugin placed before `kubernetes` or `forward` can redirect queries in unexpected ways.

6. Component interactions

```
merchant-service first call to webhooks.partner-payments.com
  -> search tries webhooks.partner-payments.com.axispay-core.svc.cluster.local
  -> NXDOMAIN
  -> tries webhooks.partner-payments.com.svc.cluster.local
  -> NXDOMAIN
  -> tries webhooks.partner-payments.com.cluster.local
  -> NXDOMAIN
  -> finally queries webhooks.partner-payments.com externally
  -> success, but slower than expected
```

That is why a hostname that is obviously external to a human can still pay an in-cluster search penalty to the resolver.

7. Enterprise example

A multinational platform team publishes a standard DNS debug container image containing `dig`, `drill`, `tcpdump`, `busybox`, Java and `grpcurl`. Their incident guide requires three proofs before escalating to the network team:

1. the live pod's `/etc/resolv.conf`;
2. a `dig +search` and `dig +nosearch` comparison;
3. the current CoreDNS Corefile.

Those three artefacts solve most incidents without leaving Kubernetes.

8. Real-world analogy

A mailroom clerk reading addresses with an internal routing habit: "try building, then floor, then department, then city" before accepting that the envelope is meant for another company. The habit is useful for local names and wasteful for already-complete external ones.

Where it breaks: human clerks can infer intent. DNS resolvers do not. They follow the rules in `resolv.conf` literally.

9. Best practices

Practice	Reason
Use a standard debug image with `dig`, `nslookup` and `getent`	Saves time mid-incident
Read `/etc/resolv.conf` first	Many DNS mysteries are actually search-list or policy issues
Compare `name` versus `name.` for external hosts	Isolates `ndots` and search-domain overhead
Check CoreDNS logs before restarting it	Restarts hide evidence
Inspect the Corefile for `rewrite`, `forward` and stub domain entries	Custom rules cause the strangest incidents
Keep a documented egress policy for DNS	Default-deny failures otherwise resemble outages

When external hostnames are configured in Java properties, consider storing them already fully qualified with a trailing dot if the client library accepts it.

10. Common mistakes

Mistake	Symptom
Declaring DNS broken without testing from a pod	Debugging the wrong layer
Using only `nslookup`	Misses libc behaviour that real apps use
Restarting CoreDNS first	Evidence disappears and the real bug survives
Ignoring Corefile rewrite rules	One hostname resolves wrongly, all others fine
Forgetting external names also hit the search list	Slow first connection to partners
Lowering `ndots` cluster-wide impulsively	Fixes one app, surprises many others

11. Security considerations

Threat	Control	Residual risk
Debug pods abused for lateral reconnaissance	Restrict who can launch them	Incident responders still need access
Malicious Corefile rewrite	RBAC and change review on CoreDNS ConfigMap	A privileged operator can still alter it
External DNS exfiltration	Egress policy and approved upstream resolvers	Necessary external resolution remains open
Debug commands leaking internal names into tickets	Sanitise outputs before sharing widely	Operational overhead

12. Performance considerations

- Search-domain expansion adds latency that appears only on cache misses or first connections.
- `getent hosts external.example.com.` is often the cleanest way to prove the trailing-dot benefit.
- Excessive failed lookups increase CoreDNS load even when applications eventually succeed.
- Lowering `ndots` per-pod with `dnsConfig` can help targeted workloads without changing the whole cluster.

A pod-specific override looks like this:

```
spec:
  dnsConfig:
    options:
      - name: ndots
        value: "2"
```

That is safer than surprising the whole platform unless you truly own all workloads.

13. High availability

A proper DNS debugging posture improves HA because it shortens time to innocence. If CoreDNS has two healthy replicas and search-domain expansion explains the latency, you avoid an unnecessary resolver restart in the middle of an incident. Stability often comes from *not* touching the wrong component.

14. Disaster recovery

CoreDNS itself is stateless, but its ConfigMap is not. Back up the Corefile and restore it exactly. During DR validation, test both an internal Service name and one approved external hostname from an application namespace. That catches missing upstream forwarders and accidental rewrite rules immediately.

15. Monitoring

Signal	Why
CoreDNS `SERVFAIL` / `NXDOMAIN` rates	Sudden spikes show lookup pathology
External first-connection latency from apps	Exposes `ndots` pain
CoreDNS pod restarts	Resolver instability
Drift in the CoreDNS ConfigMap	Rewrite or forwarder changes
Ratio of internal to external lookup volume	Unexpected search amplification

16. Troubleshooting

Symptom	Cause	Command	Fix
`UnknownHostException` for all Services	DNS egress blocked or CoreDNS down	`nslookup kubernetes.default` from a pod	Allow DNS or restore CoreDNS
External hostname slow only on first call	`ndots` search expansion	`time getent hosts host`; compare with `host.`	Use trailing dot or lower `ndots`
One hostname resolves to the wrong target	Corefile rewrite rule	`kubectl get cm coredns -n kube-system -o yaml`	Fix or remove rewrite
`dig` works, app still fails	App runtime cache or different resolver path	Compare with `getent hosts` and app logs	Adjust runtime or library config
Some DNS queries time out intermittently	TCP/53 blocked by policy	`dig +tcp` from a pod	Allow TCP/53
CoreDNS healthy, still no response in one namespace	Namespace policy denies egress	Check namespace NetworkPolicies	Add DNS rule there too

Interview questions

1. **What is the first command you run when debugging Kubernetes DNS from an application namespace?** *Read `/etc/resolv.conf` from a live pod or debug pod. It tells you the nameserver, search domains and `ndots`, which explain a large fraction of DNS behaviour immediately.*
2. **Why use `getent hosts` as well as `dig`?** *Because `getent` exercises the system resolver path most applications use, while `dig` talks to DNS more directly. If `dig` works and `getent` is slow, search-list or libc behaviour is often involved.*
3. **How can an external hostname be slowed by Kubernetes search domains?** *With `ndots:5`, a non-FQDN external name is tried with each search suffix first. The resolver may burn several failed lookups before finally querying the real external name.*
4. **How do you bypass that for one hostname?** *(senior) Append a trailing dot so the resolver treats it as absolute, for example `webhooks.partner-payments.com.`. Alternatively lower `ndots` via pod `dnsConfig` if the workload justifies it.*
5. **What CoreDNS artefact must always be inspected when one hostname resolves strangely?** *(senior) The `coredns` ConfigMap containing the Corefile, especially any `rewrite`, `forward`, stub domain or custom plugin sections. Healthy pods do not guarantee correct resolver logic.*

4.10 Cross-namespace communication and placement for Java workloads

AxisPay's Java services do not stay neatly inside one namespace forever. Sooner or later a compliance boundary appears: `payment-service` remains in `axispay-core`, while `fraud-service` moves to `axispay-risk` because a different team owns the scoring models and its data retention rules. The application call graph stays the same; the operational rules change.

The first change is naming. Inside one namespace, `fraud-service` is enough. Across namespaces, it is wrong or ambiguous. `payment-service` must call:

```
fraud-service.axispay-risk.svc.cluster.local
```

Using the full FQDN is not style points here; it is the difference between a deterministic dependency and "whatever the resolver means from this namespace". For Spring Boot services, the clean pattern is to place the full service name directly in configuration:

```
axispay.fraud.base-url=http://fraud-service.axispay-risk.svc.cluster.local:8080
```

The second change is policy. When namespaces separate for RBAC or compliance reasons, NetworkPolicy must now describe traffic that crosses that boundary explicitly on **both** sides. `payment-service` needs egress to `axispay-risk`, and `fraud-service` needs ingress from `axispay-core`. If either selector is wrong, the symptom is a timeout that looks identical to an ordinary network failure.

A narrow pair of policies looks like this:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-payment-to-risk
  namespace: axispay-core
spec:
  podSelector:
    matchLabels:
      app: payment-service
  policyTypes: [Egress]
  egress:
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: axispay-risk
          podSelector:
            matchLabels:
              app: fraud-service
      ports:
        - protocol: TCP
          port: 9090
```

That policy alone is insufficient. The destination namespace still needs a matching ingress rule. Cross-namespace traffic is where teams most often remember one half of NetworkPolicy and forget the other.

Placement matters just as much for Java. Spring Boot services on JDK 17 are memory-hungry compared with tiny sidecars or short-lived utilities. If three `payment-service` replicas land on one node, they compete for heap, metaspace, JIT code cache and CPU at exactly the moment that node hits merchant traffic. The workshop already covered anti-affinity; the cross-namespace lesson is to use it deliberately for JVM-heavy workloads.

A sensible `podAntiAffinity` for `payment-service` keeps replicas apart by hostname:

```
podAntiAffinity:
  requiredDuringSchedulingIgnoredDuringExecution:
    - labelSelector:
        matchExpressions:
          - key: app
            operator: In
            values: [payment-service]
      topologyKey: kubernetes.io/hostname
```

That makes each replica pay its own node-level risk instead of concentrating heap pressure in one place. If the cluster is smaller than the replica count, switch to `preferred` rather than letting autoscaling stall at `Pending`.

For `fraud-service`, the priority is often latency rather than raw concurrency. Risk scoring is the synchronous gate in front of authorisation. If noisy batch jobs share those nodes, garbage collection pauses and CPU steal show up directly as slower payment approval. The usual answer is a dedicated node pool with a taint such as:

```
workload=axispay-fraud:NoSchedule
```

Then give only `fraud-service` a matching toleration **and** a `nodeSelector` or node affinity that attracts it there. The workshop rule still applies: a toleration removes a repellent; it does not attract by itself.

This combination is especially useful when the batch platform is run by another team. They are free to saturate their own nodes with analytics or reconciliation jobs; they do not get to co-locate with latency-sensitive scoring pods. The fraud platform becomes more predictable, which is often more valuable than a small gain in average utilisation.

There is also a failure-domain benefit. If `payment-service` is spread across nodes and `fraud-service` sits on its own tainted pool, a memory leak in payments does not evict fraud, and a runaway model refresh in fraud does not starve the payment API. Namespaces separate control planes; placement separates blast radius.

The operational pattern for cross-namespace Java workloads therefore becomes:

Concern	Rule
Service naming	Use FQDNs, always
Access control	Write explicit ingress and egress policies across namespaces
JVM spread	Anti-affinity or topology spread for heap-heavy replicas
Latency-sensitive workloads	Taints, tolerations and dedicated node pools
Team boundaries	Reflect them in namespaces, labels and selectors

Done together, these controls turn namespace separation from an organisational diagram into a technical boundary that the cluster actually enforces.

Day 4 cheat sheet

Services and DNS

```
kubectl get svc,endpointslice -A
kubectl get endpointslice -n <ns> -l kubernetes.io/service-name=<svc>

# the four forms
<svc>                                  # same namespace only
<svc>.<ns>                             # anywhere
<svc>.<ns>.svc.cluster.local           # fully qualified — use this
<pod>-0.<svc>.<ns>.svc.cluster.local   # a specific StatefulSet pod

kubectl exec <pod> -- cat /etc/resolv.conf     # ndots:5 lives here
```

Ingress

```
kubectl get ingress -A          # EMPTY ADDRESS = no controller claimed it
kubectl describe ingress <name> -n <ns>
kubectl logs -n ingress-nginx -l app.kubernetes.io/component=controller --tail=20
```

Code	Layer
404	Routing rules did not match — Ingress
502	Wrong backend port
503	No ready endpoints — Service or readiness

NetworkPolicy

```
kubectl get netpol -A
python3 platform/admin/validate/simulate-netpol.py     # 39 assertions

# test enforcement — reading YAML proves nothing
kubectl run probe -n <ns> --rm -i --restart=Never --image=busybox:1.37 -- \
  sh -c 'nc -z -w3 <host> <port> && echo ALLOWED || echo blocked'
```

Three properties: default-allow until selected · additive, no deny rule · both directions independently. **Always allow DNS egress, UDP *and* TCP on 53.**

Placement and disruption

```
kubectl get pods -o wide                # are replicas actually spread?
kubectl get pdb -A                      # ALLOWED DISRUPTIONS must be > 0
kubectl describe node <n> | grep -A5 Taints
kubectl cordon <n>; kubectl drain <n> --ignore-daemonsets --delete-emptydir-data
kubectl uncordon <n>
```

	`required`	`preferred`
Unsatisfiable	Pending **forever**	Scheduled anyway
Use for	Critical paths	Anything autoscaled

Day 4 review questions

1. Name the four networking rules Kubernetes requires of a CNI.
2. Does Kubernetes implement networking? What implements NetworkPolicy?
3. What does `ndots:5` cost, and how do you avoid it?
4. Why must a NetworkPolicy allow both UDP and TCP on port 53?
5. What is the difference between an Ingress resource and an Ingress controller?
6. `pathType: Prefix` versus `Exact` — what does each match?
7. 404, 502 and 503 from an Ingress: what does each point at?
8. Is NetworkPolicy default-allow or default-deny?
9. How do you write a "deny" rule?
10. You apply default-deny and everything breaks. Why, and what is the symptom?
11. Adding one narrow policy broke a service not mentioned in it. Explain.
12. What is the difference between a taint and anti-affinity?
13. Does a toleration attract a pod to a node?
14. Why does `required` anti-affinity silently cap an HPA?
15. Does a PDB protect against a node crash? An OOM kill? `kubectl delete pod`?
16. Why prefer `maxUnavailable` over `minAvailable` with an HPA?
17. Your PDB allows zero disruptions. What is the operational consequence?
18. How would you prove to an auditor that the DMZ cannot reach cardholder data?

Answers: `documents/assessments/answer-keys/day4-answer-key.md`

Day 4 summary

You built: two TLS-terminated hostnames reachable from outside the cluster · **22 NetworkPolicies** enforcing zero trust across four namespaces · `payment-service` replicas one per node by instruction · six PodDisruptionBudgets · four async services deployed.

You proved: the four networking rules from your own cluster · that `ndots:5` costs real round-trips · that an Ingress with no controller does nothing and says nothing · that default-deny breaks DNS first · **that the DMZ can no longer reach the payments database** — the same command that opened the day, with the opposite result · that a node drains under 40 rps with zero failed payments.

What is still missing:

Gap	Consequence	Fixed
No RBAC	Every ServiceAccount can do anything	L5.2
Pod Security not enforced at namespace level	A workload that forgets `securityContext` is still admitted	L5.1
No packaging	Deploying is 40 `kubectl apply` commands	L5.3
No metrics or dashboards	You cannot operate what you cannot see	L5.5
No log aggregation	Tracing one payment means `kubectl logs` across 15 services	L5.6
No alerts	A merchant tells you before your monitoring does — as happened all week	L5.6

Tonight (optional, 10 minutes): run `python3 platform/admin/validate/simulate-netpol.py` and read the output. Then look at `MUST_BLOCK` and ask yourself which of those you would have thought to test.